# 3.移植linux kernel和rootfs

# 全志V3S嵌入式驱动开发（移植linux kernel和rootfs）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2024-01-20 23:01:23 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量1.6k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 4

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数 2

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发 linux 运维

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文详细介绍了如何在全志V3S芯片上构建嵌入式Linux系统，包括下载并编译Linux kernel，配置rootfs，以及烧录SD卡的步骤。通过配置kernel的menuconfig，编译生成zImage和驱动模块，然后使用buildroot创建rootfs。在Ubuntu中对SD卡进行分区并构建文件系统，将编译好的文件拷贝到对应分区并启动板子，最终成功运行最小的嵌入式Linux系统。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

除了少部分嵌入式设备使用vxworks和freertos之外，大部分嵌入式都是使用linux来做基础os来使用的。linux使用场景很多，除了大家都知道的android设备、娱乐设备之外，在汽车、工控、网络交换机、服务器等领域，linux也扮演着重要的角色。所以，对于linux或者linux kernel来说，大家没必要神话它，也没有必要轻视它，就把它当成一个工具好了，该优化优化、该加驱动加驱动、该写上层软件写上层软件就行。

前面学习了uboot，板子是可以启动了，但是不能称之为一个完整的系统，因为它还缺少了两样东西，一个是linux kernel，一个是rootfs。

**不同厂家的soc，一般对uboot和kernel，他们自己都已经移植好了，用户只需要到对应的地址下载代码编译即可。** 而对于rootfs，如果本身没有特殊的需求，可以考虑直接使用别人做好的rootfs.tar，这样效率会高很多。

## 1、下载linux kernel

```
git clone https://github.com/Lichee-Pi/linux.git -b zero-4.10.y
```

不出意外，我们就可以下载到4.10.y版本的linux内核代码，首先，配置config文件

```
make ARCH=arm licheepi_zero_defconfig
```

**如果后期涉及到kernel的宏放开，务必menuconfig设置下，同时保存，**

```
make ARCH=arm menuconfig
```

接着，开始编译内核，

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```

这个时候就可以搜索发现，生成了一个zImage文件，接着继续编译设备树

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- dtbs
```

最后就是生成驱动modules，

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=out
modules
```

当然，如果需要自己把驱动放到统一的目录里面，就可以这么做，

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=out
modules_install
```

# 2、编译rootfs

关于rootfs，一般都是用buildroot进行配置编译的。之前练习的时候，下载的buildroot版本是buildroot-2017.08.tar.gz。这部分还是建议使用别人编译好的根文件系统，差别不大，编译好之后，在./output/images目录下面，会有一个rootfs.tar文件，这就是我们需要的根文件系统。在文章的最后，我们也给出了自己编译好的rootfs.tar下载地址，供大家参考。

# 3、烧入sd卡

除linux kernel编译和rootfs编译之外，另外很重要的一个部分就是sd卡烧入。所有的操作都是在ubuntu虚拟机里面完成的，有这么几个流程，

## 3.1 分区

利用ubuntu里面的fdisk进行分区，输入

```
sudo fdisk /dev/sdb
```

**分成两个区即可，第一个分区32M，剩下的空间留给另外一个分区，注意在第一个分区的最后一步输入+32M即可。**

## 3.2 在刚才创建好的分区构建文件系统

```
sudo mkfs.vfat /dev/sdb1
sudo mkfs.ext4 /dev/sdb2
```

注意第一个fs的起始sector往后靠一点，比如2048开始，这样不会和后面的uboot重合。

## 3.3 重新插拔sd卡

```
sudo sync
sudo eject /dev/sdb
```

## 3.4 拷贝文件

把zImage和sun8i-v3s-licheepi-zero.dtb拷贝到vfat分区，与此同时把rootfs.tar拷贝到ext4分区，并解压，

```
tar -xvf rootfs.tar ./
```

因为之前已经烧入过uboot，这步可以省略，当然不放心的话，可以再烧入一下，

```
sudo dd if=u-boot-sunxi-with-spl.bin of=/dev/sdb bs=1024 seek=8
```

## 3.5 重启板子

因为uboot中我们没有对代码进行修改，所以uboot启动后，需要输入下面这些命令，才能启动kernel

```
setenv bootm_boot_mode sec
load mmc 0:1 0x41000000 zImage
load mmc 0:1 0x41800000 sun8i-v3s-licheepi-zero.dtb

setenv bootargs console=ttyS0,115200 panic=5 rootwait
```

```
root=/dev/mmcblk0p2 earlyprintk rw  vt.global_cursor_default=0
bootz 0x41000000 - 0x41800000
```

# 4、效果图

之前说过，uboot启动后，如果需要进入Linux，那么需要进行一些配置，效果如下，

Open: 28e6c9f8db762bce25be7090489e7abc_MD5.png

```
U-Boot 2017.01-rc2 (Feb 15 2023 - 17:54:53 +0800) Allwinner Technology

CPU:   Allwinner V3s (SUN8I 1681)
Model: Lichee Pi Zero
DRAM:  64 MiB
MMC:   SUNXI SD/MMC: 0
*** Warning - bad CRC, using default environment

In:    serial@01c28000
Out:   serial@01c28000
Err:   serial@01c28000
Net:   No ethernet found.
starting USB...
No controllers found
Hit any key to stop autoboot:  0
=> setenv bootm_boot_mode sec
=> load mmc 0:1 0x41000000 zImage
reading zImage
3828504 bytes read in 198 ms (18.4 MiB/s)
=> load mmc 0:1 0x41800000 sun8i-v3s-licheepi-zero.dtb
reading sun8i-v3s-licheepi-zero.dtb
8585 bytes read in 25 ms (335 KiB/s)
=> setenv bootargs console=ttyS0,115200 panic=5 rootwait root=/dev/mmcblk0p2 earlyprintk rw  vt.global_cursor_default=0
=> bootz 0x41000000 - 0x41800000
## Flattened Device Tree blob at 41800000
   Booting using the fdt blob at 0x41800000
   Loading Device Tree to 42dfa000, end 42dff188 ... OK
```

CSDN @嵌入式-老费

所有命令输入完毕之后，也就是输入bootz 0×41000000 - 0×41800000后，kernel就可以开始启动了，

Open: 1007235ac547639e5ff54cca29492a16_MD5.png

```
Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 4.10.15-licheepi-zero (feixiaoxing@feixiaoxing-VirtualBox) (gcc version 6.3.1 20170404 (Linaro GCC 6.3-2017.05) ) #1 SMP Sun Apr 9 18:22:49 CST 2023
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.000000] OF: fdt:Machine model: Lichee Pi Zero
[    0.000000] Memory policy: Data cache writealloc
[    0.000000] percpu: Embedded 14 pages/cpu @c3f63000 s24716 r8192 d24436 u57344
[    0.000000] Built 1 zonelists in Zone order, mobility grouping on.  Total pages: 16256
[    0.000000] Kernel command line: console=ttyS0,115200 panic=5 rootwait root=/dev/mmcblk0p2 earlyprintk rw vt.global_cursor_default=0
[    0.000000] PID hash table entries: 256 (order: -2, 1024 bytes)
[    0.000000] Dentry cache hash table entries: 8192 (order: 3, 32768 bytes)
[    0.000000] Inode-cache hash table entries: 4096 (order: 2, 16384 bytes)
[    0.000000] Memory: 55156K/65536K available (6144K kernel code, 199K rwdata, 1388K rodata, 1024K init, 259K bss, 10380K reserved, 0K cma-reserved, 0K highmem)
[    0.000000] Virtual kernel memory layout:
[    0.000000]     vector  : 0xffff0000 - 0xffff1000   (   4 kB)
[    0.000000]     fixmap  : 0xffc00000 - 0xfff00000   (3072 kB)
[    0.000000]     vmalloc : 0xc4800000 - 0xff800000   ( 944 MB)
[    0.000000]     lowmem  : 0xc0000000 - 0xc4000000   (  64 MB)
[    0.000000]     pkmap   : 0xbfe00000 - 0xc0000000   (   2 MB)
[    0.000000]     modules : 0xbf000000 - 0xbfe00000   (  14 MB)
[    0.000000]       .text : 0xc0008000 - 0xc0700000   (7136 kB)
[    0.000000]       .init : 0xc0900000 - 0xc0a00000   (1024 kB)
[    0.000000]       .data : 0xc0a00000 - 0xc0a31e00   ( 200 kB)
[    0.000000]        .bss : 0xc0a33000 - 0xc0a73f0c   ( 260 kB)
[    0.000000] SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[    0.000000] Hierarchical RCU implementation.
[    0.000000]  Build-time adjustment of leaf fanout to 32.
[    0.000000]  RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=1.
[    0.000000] RCU: Adjusting geometry for rcu_fanout_leaf=32, nr_cpu_ids=1
[    0.000000] NR_IRQS:16 nr_irqs:16 16
[    0.000000] arm_arch_timer: Architected cp15 timer(s) running at 24.00MHz (virt).
[    0.000000] clocksource: arch_sys_counter: mask: 0xffffffffffffff max_cycles: 0x588fe9dc0, max_idle_ns: 440795202592 ns
[    0.000006] sched_clock: 56 bits at 24MHz, resolution 41ns, wraps every 4398046511097ns
[    0.000017] Switching to timer-based delay loop, resolution 41ns
[    0.000134] clocksource: timer: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 79635851949 ns
[    0.000335] Console: colour dummy device 80x30
[    0.000371] Calibrating delay loop (skipped), value calculated using timer frequency.. 48.00 BogoMIPS (lpj=240000)
```

CSDN @嵌入式-老费

因为目前驱动的内容比较少，上层软件也不多，所以不到2s就启动结束，这里输入rootfs之后，马上就进入根文件系统了，

Open: e96e6392355430048ca10bf9291ed6ae_MD5.png

```
[    0.860578] NET: Registered protocol family 17
[    0.865281] Key type dns_resolver registered
[    0.869681] Registering SWP/SWPB emulation handler
[    0.880831] usb_phy_generic.0.auto supply vcc not found, using dummy regulator
[    0.888807] musb-hdrc musb-hdrc.1.auto: MUSB HDRC host driver
[    0.894653] musb-hdrc musb-hdrc.1.auto: new USB bus registered, assigned bus number 1
[    0.905908] hub 1-0:1.0: USB hub found
[    0.909790] hub 1-0:1.0: 1 port detected
[    0.914561] using random self ethernet address
[    0.919049] using random host ethernet address
[    0.924636] usb0: HOST MAC d6:4b:01:34:97:77
[    0.928952] usb0: MAC f6:92:d8:f8:43:62
[    0.932827] g_cdc gadget: CDC Composite Gadget, version: King Kamehameha Day 2008
[    0.940405] g_cdc gadget: g_cdc ready
[    0.944423] sun6i-rtc 1c20400.rtc: setting system clock to 1970-01-01 00:02:33 UTC (153)
[    0.952753] vcc3v0: disabling
[    0.955798] vcc5v0: disabling
[    0.958769] ALSA device list:
[    0.961730]   No soundcards found.
[    0.967224] Waiting for root device /dev/mmcblk0p2...
[    1.010425] mmc0: host does not support reading read-only switch, assuming write-enable
[    1.020320] mmc0: new high speed SDXC card at address 59b4
[    1.026852] mmcblk0: mmc0:59b4 SD64G 59.4 GiB
[    1.033355]  mmcblk0: p1 p2
[    1.054171] EXT4-fs (mmcblk0p2): couldn't mount as ext3 due to feature incompatibilities
[    1.063206] EXT4-fs (mmcblk0p2): couldn't mount as ext2 due to feature incompatibilities
[    1.095580] EXT4-fs (mmcblk0p2): mounted filesystem with ordered data mode. Opts: (null)
[    1.103782] musb-hdrc musb-hdrc.1.auto: VBUS_ERROR in b_idle (80, <SessEnd), retry #0, port1 00000100
[    1.113146] VFS: Mounted root (ext4 filesystem) on device 179:2.
[    1.121865] devtmpfs: mounted
[    1.126154] Freeing unused kernel memory: 1024K
[    1.240846] EXT4-fs (mmcblk0p2): re-mounted. Opts: data=ordered
Starting logging: OK
Initializing random number generator... [    1.359752] random: dd: uninitialized urandom read (512 bytes read)
done.
Starting network: OK

Welcome to Buildroot
buildroot login: root
# uname -a
Linux buildroot 4.10.15-licheepi-zero #1 SMP Sun Apr 9 18:22:49 CST 2023 armv7l GNU/Linux
```

CSDN @嵌入式-老费

至此，一个最小的嵌入式linux系统就已经实实在在的跑起来了。

## 注：

最后为了大家使用方便，这里给出rootfs.tar资源，有小伙伴希望直接可以拿过来用的话，那么可以直接下载使用， https://download.csdn.net/download/feixiaoxing/87816983 。